

Assessing LLM-Generated Galaxy Workflows

We evaluate the ability of LLMs to generate complete and executable workflows for representative Galaxy tasks. For each workflow, we first manually construct and execute the corresponding pipeline by following the GTN tutorial, which serves as our baseline for correctness and platform compatibility. We then provide the same workflow-generation prompt to *GPT-4o*, *Gemini 2.5 Flash*, and *DeepSeek-V3* and compare their outputs in terms of correctness, completeness, tool selection, executability, and usability. When an instruction-only prompt fails to produce a fully usable workflow, we further examine whether more guided prompting strategies, such as role-based prompting or chain-of-thought prompting, improve the result.

For Workflow W1: *Identifying the exon with the highest number of SNPs on human chromosome 22* within the Galaxy SWS, we first manually construct and execute the workflow using the Galaxy platform. This was done by following the step-by-step guidance provided in the GTN tutorial. Utilizing the sample dataset provided by GTN, we determine that the exon *ENST00000253255.7_cds_0_0_chr22_46256561_r* contains the highest number of SNPs (27 in total) on human chromosome 22. Subsequently, we apply the instruction-only prompt to GPT-4o, asking it to generate the corresponding workflow steps. The model successfully produces a logically ordered Galaxy workflow that correctly identifies the same exon. Notably, while the GTN tutorial assumes that the input files are already restricted to chromosome 22, GPT-4o explicitly includes filtering steps to isolate chromosome 22 from genome-wide datasets. This reflects a more realistic real-world scenario and demonstrates the model’s capacity to generalize beyond the assumptions of curated tutorials. In terms of tool selection, we observe minor differences in the implementation details. The GTN workflow uses the *bedtools intersect intervals* tool to compute overlaps between SNPs and exons, whereas GPT-4o suggests the use of the *Join* tool to perform the same operation. Additionally, while the GTN workflow employs the *Datamash* tool to count SNPs per exon, GPT-4o proposes using the *Group* tool. Despite these differences in tool choices, the overall logic and structure of the workflow remain correct and effective.

To evaluate the performance of Gemini 2.5 Flash, we use the same instruction-only prompt used previously. The model generates a clear and well-structured workflow, offering step-by-step guidance that includes both data acquisition and filtering procedures. Similar to the GTN tutorial, Gemini 2.5 Flash recommends using the *bedtools intersect intervals* tool to compute overlaps between SNPs and exons. However, for aggregating SNP counts per exon, it opted for the *Group* tool instead of the *Datamash* tool used in GTN. The workflow also includes the use of the *Sort* tool, a common step across all three implementations: GTN, GPT-4o, and Gemini 2.5 Flash. Upon executing the workflow generated by Gemini 2.5 Flash using the same GTN dataset, we obtain the same correct result, identifying the exon *ENST00000253255.7_cds_0_0_chr22_46256561_r* with 27 SNPs, as observed in the baseline and GPT-4o workflows.

For DeepSeek-V3, we are again successfully able to generate the steps to develop a correct and executable workflow. Similar to GPT-4o, DeepSeek-V3 recommends the use of the *Intersect* tool to compute overlaps between SNPs and exon regions. Additionally, the model offers flexibility by suggesting alternative tools for certain tasks. For example, to count the number of SNPs per exon, it proposes using either the *Group* or *Count* tool, both valid and appropriate choices. For identifying the exon with the highest SNP count, it recommends the combination of *Sort* followed by *Select*, which aligns with standard Galaxy practices. Notably, DeepSeek-V3 also provides supplementary informa-

tion beyond the core task. It includes guidance on how to export, import, and execute the workflow within the Galaxy platform, along with suggestions for potential public data sources for exons and SNPs. While such details go beyond the minimal workflow requirements, they enhance usability for novice users.

Given that the instruction-only prompt produce valid and complete workflows across all three LLMs, we do not proceed with the more elaborate Role-based or CoT prompting strategies for this task. This outcome indicates that for well-defined and well-scoped problems like W1 (Galaxy), minimal prompting is sufficient to guide advanced LLMs in generating accurate and executable workflows.

While all three LLMs, successfully generate correct and executable workflows from the instruction-only prompt, Gemini 2.5 Flash stands out for its combination of technical accuracy and instructional clarity. Its response closely aligns with the GTN tutorial, uses appropriate tools, and provides a structured, step-by-step guide that balances detail with usability. Although GPT-4o produces a concise and accurate workflow with thoughtful generalizations, such as including chromosome filtering, it lacks some of the explanatory depth seen in Gemini’s response. DeepSeek-V3 offers additional contextual guidance and tool flexibility, which may benefit novice users, but it leans toward verbosity and lacks the structured clarity of Gemini. Overall, Gemini 2.5 Flash offers the most intuitive and comprehensive support for this task, making it the preferred choice for users seeking both correctness and clear instructional guidance. The LLMs generated workflows are shared in the supplementary material.

We next evaluate Workflow W2: *How to get from peak regions to a list of gene names?*. The primary objective is to identify genes that overlap with experimentally derived peak regions and to summarize their distribution across the genome. Following the GTN tutorial, we upload the dataset *GSE37268_mof3.out.hpeak.txt.gz*, obtained from the *Gene Expression Omnibus (GEO)*. To annotate the peaks, we retrieve a gene annotation file for the mouse genome from the UCSC Genome Browser and compute overlaps between the peak regions and annotated genes. Finally, we quantify the number of overlapping genes per chromosome to identify regions with enriched regulatory activity. Our analysis shows that chromosome 11 (chr11) contains the highest number of overlapping genes (2164), suggesting notable regulatory significance in this region.

Then, we begin the workflow generation by applying an instruction-only prompt to GPT-4o. The model returns a logically ordered and mostly accurate workflow consistent with common analysis practices. However, it omits a critical preprocessing step: the chromosome identifiers in the input file are represented numerically (e.g., 1, 20, 21), which are incompatible with standard UCSC annotations that use labels such as chr1, chrX, and chrY. Without normalization of these identifiers, such as prefixing with chr and mapping 20 and 21 to X and Y, the *BEDTools Intersect* operation fails to correctly associate peaks with genes, compromising result accuracy. To address this limitation, we escalate to a role-based prompt, instructing the model to act as a bioinformatics workflow developer. This approach provides the necessary context to guide GPT-4o in including the missing preprocessing step, thereby producing a more accurate and executable workflow. Following the steps, we are able to obtain the correct result from the workflow.

To evaluate Gemini 2.5 Flash’s ability to generate Workflow W2, we apply the same instruction-only prompt. Gemini responds with a comprehensive and intuitively organized step-by-step guide that demonstrates a strong understanding of both the biolog-

ical context and the technical requirements. Notably, the model includes critical preprocessing stages, such as acquiring ChIP-seq peak data from public repositories (e.g., GSE37268_mof3.out.hpeak.txt.gz) and retrieving gene annotation files from the UCSC Main Table Browser with appropriate parameterization. It accurately outlines core analytical steps using standard Galaxy tools, including *BEDTools Intersect* for overlapping genomic intervals, followed by *Cut columns* and *Select unique lines* to generate a non-redundant list of gene names. The clarity, depth, and correctness of the response make it particularly suitable for novice users and confirm the model’s capability to produce a valid workflow using minimal prompting.

Similarly, DeepSeek-V3 performs well under the instruction-only prompt, generating a clear and well-structured workflow for identifying target genes from ChIP-seq peak regions. Although its tool choices differ slightly, the methodology remains sound. For instance, DeepSeek-V3 recommends using *ChIPseeker* for annotating peaks with the nearest genes, while also suggesting *BEDTools closest* as a viable option. Importantly, unlike GPT-4o, DeepSeek-V3 explicitly acknowledges the need to normalize chromosome identifiers (e.g., converting numeric values to UCSC-style labels), ensuring compatibility with downstream gene annotation. Furthermore, the model enhances usability by including a visual workflow diagram, which helps users conceptualize the overall analysis pipeline.

In comparing the three LLMs on this task, clear differences emerge in completeness, clarity, and sensitivity to data formatting nuances. GPT-4o generates a mostly accurate and executable workflow but omits the crucial preprocessing step for chromosome label conversion. This omission necessitated an additional role-based prompt to achieve a fully functional pipeline. In contrast, Gemini 2.5 Flash offers the most technically complete and contextually aware solution, implicitly aligning data sources and addressing format compatibility through precise instructions. DeepSeek-V3 also performs commendably, offering methodologically valid alternatives, recognizing critical preprocessing needs, and improving accessibility through visual aids. Overall, Gemini delivers the most robust and reliable output, DeepSeek enhances interpretability and ease of use, while GPT-4o benefits from additional contextual scaffolding to meet expert-level expectations.

Our third workflow, W3: *Do genes on opposite strands ever overlap? If so, how often?*, explores the prevalence of gene overlaps between forward (+) and reverse (−) DNA strands, offering insights into the complexity of genomic organization. Following the GTN tutorial, we construct and execute a workflow that utilizes gene annotation data from the UCSC Table Browser. The analysis reveals that approximately 38.46% of the genes exhibit strand overlap, indicating a substantial level of strand-specific gene co-localization.

To generate this workflow using GPT-4o, we apply an instruction-only prompt. The model produces a concise and logically ordered set of instructions outlining the core steps required for workflow construction in Galaxy. Based on this guidance, we are able to successfully recreate the workflow and replicate the results, confirming the correctness and usability of the generated steps. However, the output exhibits limitations in procedural specificity. For instance, while the model accurately notes the need to separate genes by strand into two datasets (forward and reverse), it fails to detail how to perform this operation. It does not indicate that strand information is typically located in the sixth column of BED-formatted files, nor does it reference where to obtain the gene annotation dataset, an omission that could hinder users unfamiliar with genomic data sources or file conventions. These limitations underscore a broader observation: although GPT-4o’s high-level instructions are sufficient for experienced users who understand data structures

and workflows, novice users may struggle without additional domain-specific guidance. Despite this, the generated workflow was complete and executable, so we did not proceed with alternative prompting strategies such as role-based or chain-of-thought prompting for this task.

To evaluate Gemini 2.5 Flash’s performance on Workflow W3, we issue the same instruction-only prompt used previously. The model delivers a comprehensive, step-by-step guide that not only identifies the appropriate analytical tools, such as conditional filtering for strand separation and *BEDTools Intersect* for overlap analysis, but also includes explicit parameter configurations (e.g., using `c6 == '+'` to isolate forward-strand genes). In addition to the analytical workflow, Gemini provides detailed instructions on how to construct the workflow within the Galaxy interface, including drag-and-drop operations in the Galaxy workflow editor and guidance on saving and executing the pipeline. This level of procedural depth and platform-specific instruction makes the output highly accessible to users with limited experience in Galaxy.

DeepSeek-V3 also performs well under the same instruction-only prompt, producing a correct and well-structured workflow for identifying overlapping genes on opposite DNA strands. The model suggests a methodologically sound approach that includes optional sorting steps, strand separation via filtered expressions, and intersection analysis. It specifies key technical details, such as enforcing a minimum one bp overlap, and outlines optional post-processing steps, including counting overlaps or isolating unique gene pairs for further analysis.

Across all three models, we observe the successful generation of valid Galaxy workflows from the instruction-only prompt. GPT-4o provides a concise and logically structured outline but lacks critical implementation details such as the specific column (typically column 6 in BED format) used to represent strand information and the strandedness options required during intersection. These omissions could hinder less experienced users. In contrast, Gemini 2.5 Flash delivers the most comprehensive and accessible solution, with clear explanations of strand-specific filtering, intersection logic, and platform-level guidance for workflow execution. DeepSeek-V3 offers a robust and technically accurate workflow, enriched with optional enhancements that improve usability and extend analytical depth. Overall, Gemini 2.5 Flash emerges as the most effective model for supporting both novice and intermediate users, balancing accuracy, clarity, and practical usability in workflow construction.

We next examine Workflow W4: *How can the quality of NGS raw data be assessed, what parameters should be checked, and how can the quality of the dataset be improved?* This workflow addresses a fundamental preprocessing step in next-generation sequencing (NGS) analysis: assessing and improving the quality of raw *FASTQ* files before downstream applications. Following the GTN tutorial, we construct a workflow that begins by inspecting raw sequencing reads and evaluating quality metrics using tools such as *FastQC* and *FASTQE* (for short Illumina reads), or *NanoPlot* and *PycoQC* (for long Nanopore reads). These tools report per-base quality scores, GC content, adapter contamination, sequence duplication levels, read length distributions, and run-level metrics. The workflow then incorporates trimming and filtering steps using tools like *Cutadapt* and *Fastp* to remove low-quality bases, adapter sequences, ambiguous reads, and short fragments. Paired-end data handling is explicitly supported, and the final results are summarized using *MultiQC*. The tutorial emphasizes the iterative nature of quality control: *assess* → *clean* → *re-assess*.

To generate the workflow using LLMs, we apply the instruction-only prompt to GPT-

4o. The resulting workflow closely mirrors the core structure of the GTN pipeline, adhering to the *assess* $\rightarrow$ *clean* $\rightarrow$ *re-assess*. GPT-4o outlines the primary steps effectively, recommending *FastQC* for initial assessment, *Cutadapt* for trimming, and *MultiQC* for summarizing results. However, its workflow is streamlined and focuses primarily on short-read Illumina data, lacking the platform-specific enhancements present in the GTN tutorial. However, GPT-4o produces a minimal yet functional workflow suitable for general Illumina data, prioritizing simplicity and automation.

The workflow generated by Gemini 2.5 Flash closely mirrors the core structure of the GTN Quality Control tutorial, adhering to the standard *assess* $\rightarrow$ *clean* $\rightarrow$ *re-assess* paradigm. It utilizes essential tools such as *FastQC* for both pre- and post-trimming quality assessment, *MultiQC* for summarizing quality reports, and trimming tools like *Trimmomatic* or *fastp* to remove low-quality bases and adapter sequences. While well-suited for short-read (Illumina) data and effective for general QC workflows, Gemini's output remains focused on commonly used tools and does not incorporate platform-specific enhancements. In contrast, the GTN tutorial extends its coverage to include *NanoPlot* and *PycoQC* for long-read Oxford Nanopore data, and offers additional options like *Cutadapt*, making it more inclusive for a broader range of sequencing technologies.

DeepSeek-V3 also aligns closely with the conceptual structure of the GTN Quality Control tutorial, particularly in its adoption of the *assess* $\rightarrow$ *clean* $\rightarrow$ *re-assess* paradigm and its use of core tools such as *FastQC*, *MultiQC*, and trimming utilities like *Cutadapt*, *Trim Galore*, *Trimmomatic*, and *fastp*. The model introduces optional filtering tools such as *PRINSEQ* and *BBDuk*, and allows users to choose between *Cutadapt* and *Trim Galore!* for adapter removal. Additionally, it provides detailed implementation guidance, including parameter recommendations, memory usage optimization, paired-end read handling instructions, and version tracking to promote reproducibility. The inclusion of a workflow diagram further enhances usability and instructional value.

When comparing all three LLMs on W4, each successfully generates a valid Galaxy-based QC workflow, but they vary in depth and adaptability. GPT-4o provides a simplified yet accurate representation of the QC process using a minimal toolset, making it suitable for rapid assessment of Illumina datasets but lacking in tool diversity and parameter specificity. Gemini 2.5 Flash improves on this by offering alternative tools, implementation context, and better instructional clarity, which benefits novice users. DeepSeek-V3, however, produces the most comprehensive and customizable workflow, capturing both the conceptual rigor and practical nuances of multi-platform NGS data quality control. Its extended toolset and configuration guidance make it particularly well-suited for diverse sequencing protocols and robust real-world usage.

Our final workflow, W5: *Which genes are on a draft bacterial genome, and which other genomic components can be found on a draft bacterial genome?*, focuses on systematically annotating an assembled bacterial genome to identify and characterize its genetic content. The workflow begins by accepting a draft genome assembly in FASTA (contigs) format and uses *Bakta* to predict and annotate core genomic features, including coding sequences, tRNAs, rRNAs, and other functional elements. It proceeds to detect plasmid replicons using *PlasmidFinder*, insertion sequences with *ISEScan*, and integron structures with *IntegronFinder*. The resulting annotations are reformatted into *GFF3* format, and the final step involves visualizing the annotated genome interactively using *JBrowse*. Following the GTN tutorial, we develop the workflow and transform raw genome assemblies into richly annotated and visually interpretable bacterial genomes following the GTN instructions.

To evaluate the ability of LLMs to generate this workflow, we provide an instruction-only prompt (W5, Table ??). When applied to GPT-4o, the model generates a logically structured workflow that addresses the core objectives, including gene prediction, detection of plasmids, integrons, and insertion sequences, and genome visualization. However, several deviations from the GTN tutorial are evident. Notably, GPT-4o recommends *Prokka* for gene annotation instead of *Bakta*, which is the preferred tool in the GTN due to its active maintenance and more comprehensive annotation features. While the inclusion of *PlasmidFinder* and *IntegronFinder* aligns with the GTN protocol, the model’s suggestion to use tools like *ISMMapper* or *MobileElementFinder* for insertion sequence identification introduces challenges as these tools are not readily available within standard Galaxy instances or the Galaxy ToolShed, limiting practical reproducibility. Additionally, GPT-4o recommends *Trackster* for visualization, whereas the GTN workflow uses *JBrowse*, which offers better integration for multi-track genomic visualization in Galaxy. Another limitation of the GPT-4o-generated workflow is the omission of key preprocessing steps emphasized in the GTN tutorial, such as renaming FASTA headers to conform with tool input requirements, and the absence of instructions for combining and formatting output files prior to visualization. These gaps underscore the challenge of translating high-level workflow synthesis into fully executable, platform-aware implementations. While GPT-4o captures the overarching intent of the annotation pipeline, it lacks the implementation-specific knowledge needed to produce a robust and directly usable Galaxy workflow.

Since the workflow generated by GPT-4o in response to the instruction-only prompt included tools not available within the Galaxy platform, such as *ISMMapper*, *MobileElementFinder*, and *Trackster*, we escalate to a role-based prompt (W5, Table ??). The goal is to assess whether providing a more context-aware prompt can guide the model to recommend Galaxy-compatible tools and generate a workflow that adheres more closely to the platform’s actual capabilities. We observe that role-based workflow uses the *Bakta* tool for comprehensive genome annotation. It begins with uploading an assembled FASTA file and utilizes *Bakta* to predict genes and functional elements, generating standardized outputs such as GFF3, GenBank, and FASTA files. However, the GPT-4o-generated workflow incorporates an evaluation step using QUAST (and optionally BUSCO) to assess annotation quality and completeness. It also introduces post-processing steps such as GFF3 sorting and genome visualization via *Trackster*. Despite this conceptual extension, we notice that certain tools proposed here, such as *Trackster* for interactive visualization and *GFF3sort* for annotation formatting, are currently not available in Galaxy instances. Thus, we are unable to develop the workflow.

We then proceed with the CoT prompting strategy to further guide GPT-4o in generating a bacterial genome annotation workflow (W5, Table ??). This prompt encourages the model to reason through the task step by step, with the aim of producing a more complete and contextually grounded workflow. While the CoT-generated workflow successfully captures the overarching goals of bacterial genome annotation, namely, structural and functional annotation, quality assessment, and preparation of outputs for visualization, it diverges in several important ways from community-curated standards such as those outlined in the GTN tutorial. Both workflows correctly employ *Bakta* as the core annotation engine. However, the CoT-generated workflow introduces *BUSCO* for completeness evaluation, a widely used tool in general genome annotation but not included in the GTN protocol for bacterial genome workflows. More significantly, the GTN tutorial emphasizes a broader approach to structural annotation by incorporat-

ing specialized tools such as *PlasmidFinder*, *IntegronFinder*, and *ISEScan*, which detect plasmids, integrons, and insertion sequences, respectively, elements that are essential for accurately characterizing mobile genetic elements in bacterial genomes. The omission of these tools from the generated workflow leads to a structurally incomplete annotation. Additionally, the workflow proposes the use of formatting and indexing tools such as *GFF3sort*, *GFF3 to GenBank*, and *SAMtools faidx* for organizing annotation outputs and enabling visualization. These tools, however, are not available within the Galaxy platform. In summary, while the generated workflow aligns with the conceptual goals of bacterial genome annotation, it contains key deviations, specifically, the exclusion of critical structural annotation tools and the suggestion of tools not available in Galaxy. As such, the GPT-4o-generated workflow is partially correct but cannot be considered fully usable without further refinement.

We then move to Gemini 2.5 Flash, where we provide the same instruction-only prompt used in the previous test. We observe that Gemini 2.5 Flash is able to return a well-structured, comprehensive set of steps along with relevant parameter details. It correctly identifies and integrates all the necessary tools for structural and functional annotation, including additional conversion and visualization steps. Notably, all suggested tools are available within the Galaxy platform, making the workflow immediately executable without requiring external scripting or unsupported components. For Gemini, the instruction-only prompt is sufficient to generate a complete, accurate, and Galaxy-compliant bacterial genome annotation workflow.

The workflow generated by DeepSeek-V3 using the same prompt demonstrates a generally accurate understanding of the bacterial genome annotation process, yet diverges from the GTN standard in several key areas. For gene prediction, DeepSeek-V3 recommends *Prodigal*, a well-known tool for prokaryotic CDS identification. For functional annotation, it suggests *EggNOG-mapper* or *BLAST+*, both of which are valid in general but require manual output conversion to GFF3, unlike the GTN tutorial’s use of *Bakta*, which integrates both gene prediction and functional annotation while producing standardized outputs natively compatible with downstream tools like *JBrowse*. On the other hand, the tools suggested by DeepSeek-V3 for structural annotation *PlasmidFinder*, *IntegronFinder*, and *ISEScan*, are consistent with those used in the GTN tutorial, indicating a correct understanding of the critical elements required to capture plasmid-borne features and mobile genetic elements. However, for non-coding RNA prediction, DeepSeek-V3 proposes the use of *Infernal*, a powerful RNA homology search tool, which is unfortunately not available on Galaxy, again highlighting a case of tool hallucination. Despite these issues, DeepSeek-V3 correctly outlines the merging of GFF3 outputs and the use of *JBrowse* for genome visualization, aligning well with the GTN approach in terms of output integration and browser-based annotation exploration.

In comparison to GPT-4o and Gemini 2.5 Flash, DeepSeek-V3 performs moderately well. Like GPT-4o, it hallucinates the availability of tools not supported in Galaxy, reducing practical executability. Gemini, by contrast, stands out for producing a structurally complete and Galaxy-native workflow using instruction-only prompt. While DeepSeek-V3 captures the correct conceptual flow and covers all major analytical components, its output is hindered by limited platform awareness and reliance on tools outside the Galaxy environment. In summary, DeepSeek-V3 offers strong logical grounding but lacks the operational precision demonstrated by Gemini.

Overall Summary for Galaxy Workflows

Among the three LLMs, Gemini 2.5 Flash consistently performs best, producing complete, accurate, and Galaxy-compliant workflows without requiring additional prompting. Its outputs demonstrate strong contextual understanding, correct tool selection, detailed parameter guidance, and seamless alignment with Galaxy's capabilities, making it particularly effective for both novice and intermediate users. DeepSeek-V3 shows a solid grasp of workflow logic and offers methodologically valid suggestions, including flexible tool alternatives and usability enhancements, but is limited by occasional tool hallucinations, recommending software not available within Galaxy, which undermines executability. GPT-4o, while concise and easy to follow, often omits critical steps or recommends unsupported tools, requiring further prompting to reach usability. Overall, Gemini stands out for its balance of technical completeness, clarity, and platform awareness, making it the most reliable LLM for automated Galaxy workflow development.